EXPENSE TRACKER PROJECT

Putting the whole chapter to work

THE PLAN

One small program you might actually use

Log expenses one at a time - category, amount, whether you have a receipt - and track:

- a running **total** spent
- a **bitmask** of which categories were used
- a **transaction id** that keeps climbing across runs

Then save a ledger to disk, the same file-persistence idea chapter 3 ended on.

CHOOSING TYPES, FOR REAL

Every type decision from this chapter, at once

VARIABLE	TYPE	WHY (LECTURE)
`totalSpent`	`double`	fractional currency (4.4)
`categoriesUsedMask`	`int`	packed bits (4.2)
`expenseCount`	`short`	small session counter (4.3)
`nextTransactionId`	`long long`	climbs across months (4.3)
`hasReceipts`	`vector<bool>`	yes/no per expense (4.5)
`categories`	`vector<char>`	one-letter codes (4.6)

LOADING THE LEDGER

Picking up where you left off

```
std::ifstream ledgerIn(ledgerFileName);
if (ledgerIn) {
    long long savedNextId{};
    double savedTotal{};
    if (ledgerIn >> savedNextId >> savedTotal) {
        nextTransactionId = savedNextId;
        totalSpent = savedTotal;
    }
}
```

Same `std::ifstream` idea from chapter 3, reading two values back instead of a list of names.

BUILDING THE BITMASK

One bit set per category used

```
switch (category) {
    case 'F': categoriesUsedMask |= 0b0001; break;
    case 'T': categoriesUsedMask |= 0b0010; break;
    case 'E': categoriesUsedMask |= 0b0100; break;
    case 'O': categoriesUsedMask |= 0b1000; break;
}
```

```
std::println("Categories used (binary): {:#06b}", categoriesUsedMask);
```

THE BUG, ON PURPOSE

Narrowing damage, not just a warning

```
int badTotal = totalSpent; // narrowing: fractional cents are silently lost
std::println("(Demonstration only) totalSpent as int: {}", badTotal);
```

Storing a running total in an int "because it's just a total" silently drops every cent. Braced initialization would have refused to compile this - a plain assignment lets it through. This is 4.7's narrowing warning, shown doing real damage to a ledger.

SAVING BACK

The ledger survives the program

```
std::ofstream ledgerOut(ledgerFileName);
ledgerOut << nextTransactionId << " " << totalSpent << "\n";
```

Next run's `promptCategory` / `promptAmount` / `promptHasReceipt` loop picks up right where this one left off.

CHAPTER WRAP-UP

What this program actually needed

NEED	C++ TYPE/TOOL
A running transaction id	`long long`
Active spending categories	bits packed into an `int`
Expenses logged today	`short`
Lifetime cents processed	`long long`
How much an expense cost	`double`
Whether you're over budget	`bool`
A spending category code	`char`

IN YOUR TOOLBOX

- **`std::print` / `std::println`** with **`{:#x}`/`{:#b}`** - print any integer in whichever base is useful, no stream state to manage
- **Digit separators** (**`1'000'000`**) - readable at a glance
- **`<cstdint>` fixed-width types** - an exact, portable bit width
- **Brace initialization (`{}`)** - refuses narrowing conversions, used everywhere a variable is declared

DEBUG IT

Step through one full expense

Breakpoint at the top of the `while (true)` loop body - step through one full iteration (`promptCategory` → `promptAmount` → `promptHasReceipt` → the bitmask `switch`) and watch every variable this chapter introduced update together in your IDE's locals view.